

Integration of the FINS Protocol into the Open-Source HMI System FUXA for Omron PLCs

DENNY CHRISNANDA
Computer Science Department
Binus Online Learning
Bina Nusantara University
denny.chrisnanda@binus.ac.id

FRANS SEBASTIAN
Computer Science Department
Binus Online Learning
Bina Nusantara University
frans.sebastian@binus.ac.id

GILANG HANSAGITA
Computer Science Department
Binus Online Learning
Bina Nusantara University
gilang.hansagita@binus.ac.id

Abstract

Industrial automation requires control systems that are adaptive, efficient, and easily integrated across production environments. Programmable Logic Controllers (PLCs) remain central to such systems due to their real-time control capabilities and high reliability. At PT XYZ, Omron PLCs are widely deployed and communicate using the Factory Interface Network Service (FINS) protocol. However, most open-source Human–Machine Interface (HMI) platforms, including the lightweight and web-based FUXA system, do not natively support FINS, limiting their applicability in Omron-based industrial settings. This study addresses this limitation by developing and integrating a FINS communication module into the FUXA HMI. The research methodology comprises a literature review, analysis of FINS communication architecture, module implementation, and performance evaluation. Experimental results demonstrate that the proposed module enables stable data read/write communication with low latency and accurate process visualization corresponding to real PLC conditions. The proposed solution provides a license-free, customizable, and web-based HMI capable of direct communication with Omron PLCs without additional middleware. The findings contribute to the advancement of open-source industrial automation technologies and support ongoing digital transformation initiatives within manufacturing environments.

Keywords—FINS, FUXA, HMI, PLC Omron, Industrial Automation.

1. INTRODUCTION

Programmable Logic Controllers (PLCs) are fundamental components in industrial automation, providing reliable real-time control with relatively simple programming requirements [1]. At PT XYZ, a multinational automotive component manufacturer, Omron PLCs dominate the production environment; internal records indicate that over 90% of production machines rely on Omron PLCs as their primary control unit. The widespread use of Omron devices in manufacturing highlights the need for robust interoperability between PLCs and modern automation software.

Human–Machine Interfaces (HMIs) are essential for enabling operator interaction with PLC-controlled processes by providing visualization, interactive control, and operational monitoring [2]. While commercial HMI/SCADA solutions offer mature features for monitoring, analytics, and alarm management that reduce downtime and support quality assurance [3], they often impose licensing costs, restrict customization, and expose organizations to vendor lock-in [4, 5]. These

limitations have motivated the adoption of open-source HMI/SCADA platforms that deliver greater flexibility, lower total cost of ownership, and the ability to tailor integrations to site-specific requirements [6].

FUXA is a lightweight, web-based open-source HMI/SCADA platform that supports common industrial protocols (e.g., Modbus, OPC UA, MQTT) and is implemented using modern web technologies such as Node.js and Angular [7, 8, 9]. Despite its extensibility and suitability for remote and IoT-enabled deployments, FUXA currently lacks native support for Omron’s Factory Interface Network Service (FINS) protocol. FINS—supporting both UDP and TCP transports—remains a prevalent protocol in Omron-based installations and provides flexible addressing and memory-area access required for efficient PLC–HMI communication [10, 11].

The absence of FINS support in widely used open-source HMIs creates a practical barrier for facilities that wish to migrate away from proprietary HMI solutions without sacrificing compatibility with existing Omron PLC fleets. This research addresses this problem by focusing on two core questions: (1) How

can FINS protocol support be seamlessly integrated into an HMI system, such as FUXA, with functional parity to existing protocols like Modbus? (2) How can the read/write operations for data via the FINS protocol be stabilized to ensure correct, real-time visualization on the User Interface (UI)? To address these questions, this study proposes the design and implementation of a FINS communication module for the FUXA platform. The objectives are to: (1) integrate FINS protocol support into FUXA with functional parity to established protocols such as Modbus, and (2) validate stable data read/write operations and accurate real-time visualization on the user interface. By enabling direct interoperability between FUXA and Omron PLCs, the proposed solution aims to offer a flexible, cost-effective, and vendor-independent HMI alternative suitable for digital transformation initiatives in manufacturing environments.

2. RELATED WORKS

The industrial automation sector has witnessed increased adoption of open-source HMI/SCADA systems as viable alternatives to commercial platforms. This shift stems from practical considerations including cost reduction, customization flexibility, and vendor independence. Multiple research initiatives have explored open-source SCADA implementations across diverse industrial applications.

Uddin et al.[12] implemented a SCADA solution for solar-powered reverse osmosis plants utilizing Node-RED, Grafana, and InfluxDB components. The study validated that community-driven SCADA architectures offer cost-effective deployment options while maintaining operational reliability. Similarly, Omidi et al.[13] developed a Node-RED-based modular architecture for renewable energy monitoring systems. Their implementation confirmed that web-based technologies can support real-time industrial monitoring with minimal hardware requirements.

Research in power system applications includes the work by Almas et al. [14], who evaluated open-source SCADA integration with Phasor Measurement Units using DNP3 protocol. The study identified specific protocol limitations, particularly in polling mechanisms, that affect system responsiveness in large-scale grid applications.

Industrial cybersecurity research has also leveraged open-source HMI platforms. Salazar et al. [15] developed ICSNet, a honeynet supporting multiple industrial protocols including Modbus TCP, IEC-104,

and EtherNet/IP. The system utilized FUXA HMI for visualization, validating its effectiveness in cybersecurity applications beyond traditional HMI roles.

Web-based SCADA applications in smaller automation contexts have been explored by Hazdi et al.[16], who implemented home automation systems using PLC-OPC integration. This work established the viability of web technologies for residential monitoring applications. Additionally, Thushara[7] documented FUXA platform deployment on edge computing devices with Modbus and MQTT protocols, confirming its adaptability for modern IoT-enabled industrial systems.

Current open-source SCADA research primarily focuses on established protocols including Modbus, OPC UA, and MQTT. Notably absent from existing literature is consideration of the Factory Interface Network Service (FINS) protocol, despite its widespread use in Omron PLC environments. This represents a significant gap in open-source HMI/SCADA protocol support for Omron-based industrial installations.

This research addresses the identified gap through development and integration of a FINS communication module within the FUXA HMI platform. The implementation extends FUXA's protocol capabilities to support direct communication with Omron PLCs. Given the prevalence of Omron equipment in manufacturing facilities like PT XYZ, this enhancement expands the practical deployment scope of open-source HMI/SCADA solutions in Omron-centric industrial environments.

3. METHODOLOGY

3.1. Research Overview

This research follows an experimental development methodology aimed at extending the open-source HMI platform FUXA by integrating support for the Omron FINS protocol. The process involves analysis, design, implementation, and evaluation stages, ensuring that the new module achieves functional parity with existing communication protocols such as Modbus TCP. The entire workflow is illustrated in Figure 1.

3.2. System Architecture

The developed system consists of three main layers: (1) the PLC device layer, (2) the communication middleware layer (Node.js backend), and (3) the visualization layer (Angular frontend). The FINS connector is implemented as a new device driver named *DeviceFins*, operating in the backend of FUXA. This

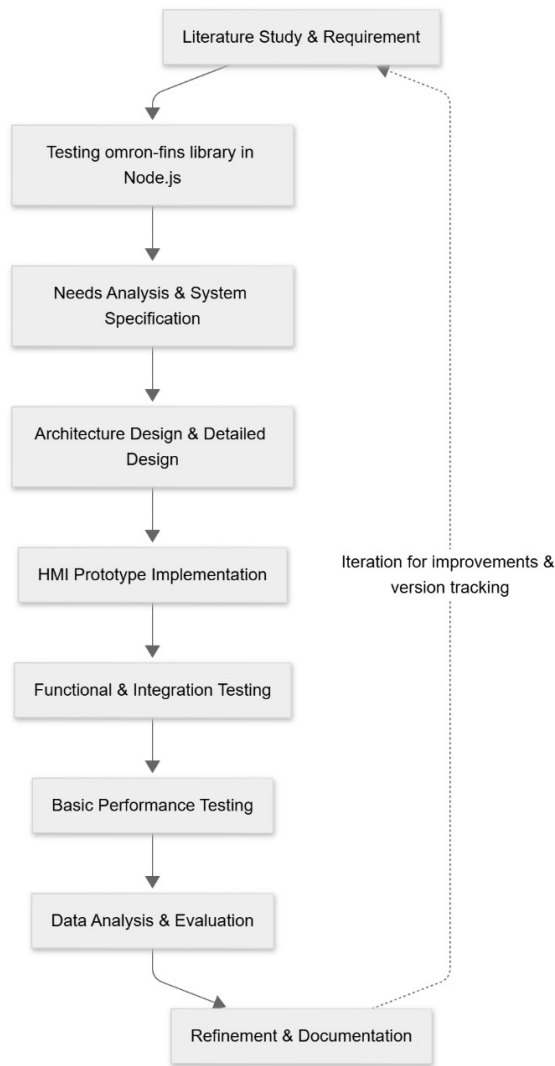


Figure 1: Research workflow for FINS protocol implementation in FUXA

module manages socket-based communication through UDP or TCP and handles read/write requests to Omron PLC memory areas such as DM, CIO, and W.

Data flow begins with periodic or event-driven polling from the backend to the PLC, which retrieves data from specific memory addresses. These values are then propagated to the frontend through the system's existing event emitter (`device-value:changed`) to update the HMI interface in real time. A simplified overview of this architecture is shown in Figure 2.

3.3. Development Tools and Environment

The implementation and testing were conducted using the following environment:

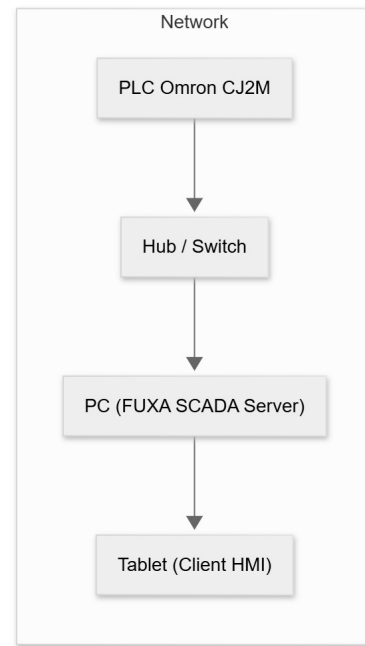


Figure 2: Architecture of the developed HMI-PLC communication system

- **Programming Languages:** JavaScript (Node.js) and TypeScript (Angular).
- **HMI Platform:** FUXA v1.2.13 (open-source web-based SCADA system).
- **PLC Device:** Omron CJ2M-CPU34.
- **Operating System:** Windows 11.
- **Network Analysis Tools:** Wireshark for packet inspection and validation.

3.4. Implementation Procedure

The implementation followed these main stages:

1. **Protocol Study and Requirements Analysis:** The FINS command structure, addressing format, and communication flow were analyzed based on Omron's official documentation.
2. **Driver Design and Coding:** The `DeviceFins` class was created to handle connection management, read/write functions, automatic reconnection, and event handling.
3. **Frontend Integration:** Configuration options such as IP address, port, DA1, and SA1 were added to FUXA's web configuration UI.
4. **Testing and Debugging:** Unit tests and integration tests were performed using both PLC simulators and physical Omron PLCs.

3.5. Testing Method

Performance evaluation was conducted using a load test script written in Node.js. The script generated 1000 consecutive read requests to PLC memory address D100 with a concurrency level of 10. Metrics collected during the test include:

- Average read/write latency (ms)
- Success rate of communication (%)
- CPU and memory utilization of the FUXA server
- Stability of reconnection handling during simulated disconnections

The results of these measurements were analyzed statistically to evaluate whether the implemented connector meets the performance characteristics comparable to Modbus TCP communication.

3.6. Evaluation Criteria

The evaluation focused on two primary objectives aligned with the research goals:

1. **Functional Equivalence:** Verifying that the implemented FINS connector supports key HMI functions—connectivity, data reading, data writing, and visualization—equivalent to existing Modbus modules.
2. **Stability and Responsiveness:** Measuring whether the FINS communication remains stable under repetitive load and can maintain real-time data visualization without significant latency or data loss.

Table 1: *omron-fins trial environment configuration*

Component	Specification
PLC	Omron CJ2M-CPU34
Client System	Intel i7-11th Gen, 16GB RAM, Windows 11
PLC Address	192.168.0.1
Client Address	192.168.0.234
Communication	FINS over UDP, port 9600
Library	omron-fins (Node.js)
Runtime	Node.js v18 LTS

The library was installed via `npm` and validated using a stress test script that issued 1 000 read requests to address D100 with a concurrency level of 10. Network traces were captured using Wireshark to confirm frame structure and response codes.

Key observations from the library tests are:

- Captured responses consistently contained Command Code 0101 (Memory Area Read) and Response Code 0000 (no error), with incrementing Service IDs indicating unique matching responses.
- The library handled read operations reliably under the configured stress: average response latency was observed below 50 ms and the library exhibited correct framing behavior according to Omron FINS specification.
- No installation or dependency errors were encountered during setup; the library is suitable as a communication basis for further integration into the HMI system.

4. RESULTS AND DISCUSSION

4.1. Evaluation of the `omron-fins` Library

The study began with an evaluation of the `omron-fins` Node.js library to verify basic FINS communication (read/write) between a PC and an Omron CJ2M PLC via UDP. The experimental setup is summarized in Table 1.

4.2. System Configuration and Integration

The FINS connector was integrated into the FUXA server under the directory `server/runtime/devices/fins`. The deployed testbed topology comprises one Omron CJ2M PLC, one PC acting as HMI server, and a tablet as the client (see Fig. 3). Typical connector parameters are listed in Table 2.

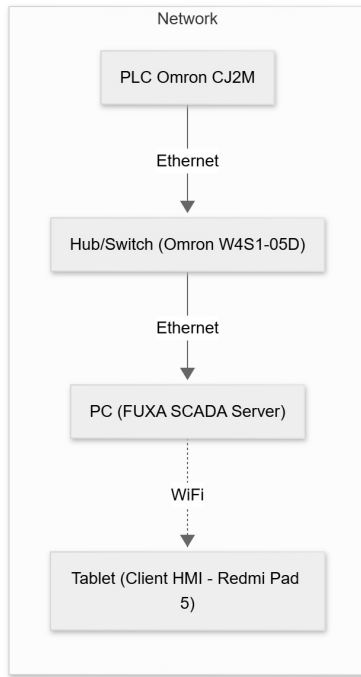


Figure 3: System topology of the FUXA HMI with the FINS connector

Parameter	Configuration Value
PLC IP Address	192.168.11.1
Communication Port	9600
Transport Protocol	UDP
Source Node (SA1)	234
Destination Node (DA1)	1
Data Polling Rate	200 ms

Table 2: FINS connection configuration parameters

After connector installation, initial verification was performed by issuing a single read to memory address D100. Successful read/response without timeout confirmed that the connector was operational.

4.3. Implemented Functionality

The implemented FINS connector provides the following capabilities:

1. Transport-level connectivity (UDP and TCP) using the `omron-fins` library.
2. Periodic polling per device with the connector only reporting `connect-ok` after a successful read cycle.
3. Write (set) operations initiated from the UI and executed on the PLC in real time.

4. Event emission to FUXA runtime (device-value:changed, device-status:changed, device-error).
5. Frontend integration: device configuration form and tag editing dialog for FINS-specific parameters (SA1, DA1, memory area, address, data type).

Representative UI screens is shown in Figs. 4.

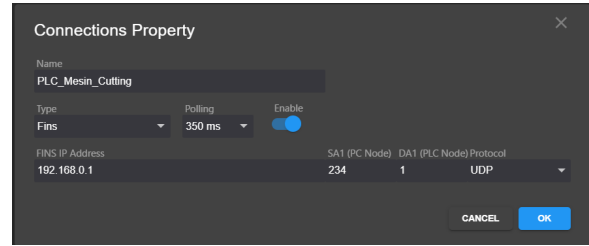


Figure 4: UI Result — Device Settings: configuration form for FINS device (IP, port, SA1, DA1, protocol).

After integrating the FINS connector module into both the backend and the HMI interface, a realtime communication test was conducted to verify bidirectional data transfer between the HMI system and the Omron PLC. Fig. 5 presents the HMI interface during the execution of the test.

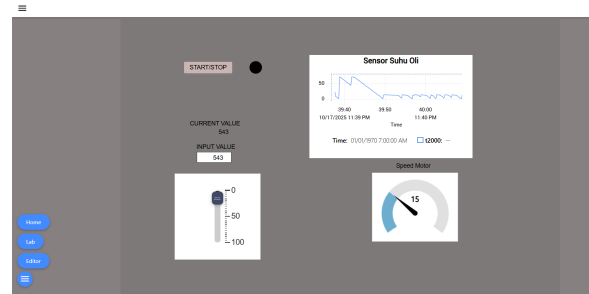


Figure 5: HMI demonstration connected to the PLC in realtime.

The HMI display comprises several key components:

- **Real-time trend (oil temperature sensor):** continuously visualizes temperature measurements acquired from the PLC via the FINS protocol.
- **Slider and numeric input field:** enable operators to write updated setpoint values to specific PLC memory addresses, with changes immediately reflected on the HMI.
- **Motor speed gauge:** presents the corresponding PLC variables through an analog-style indicator, illustrating responsive and intuitive feedback for speed changes.

Experimental measurements indicate that PLC-to-HMI data updates occur within <100ms, demonstrating that the implemented FINS connector achieves low-latency and stable real-time communication performance.

Beyond real-time monitoring and control, the system also integrates an alarm mechanism to identify abnormal operating conditions. As illustrated in Fig. 6, an alarm is raised when the oil temperature exceeds the maximum threshold configured in the PLC, allowing timely operator intervention.

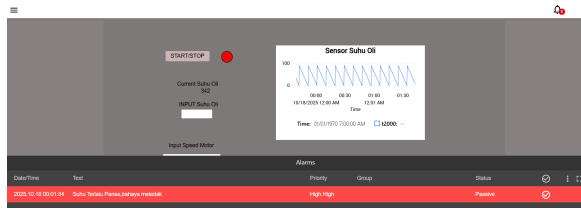


Figure 6: Real-time alarm view highlighting when the oil temperature rises above the defined safety limit.

The alarm subsystem operates in realtime with the following characteristics:

- **Continuous monitoring:** PLC tag values are sampled periodically and compared against configured thresholds.
- **Alarm visualization:** active alarms are shown in red with *High-High* priority to make critical states easy to notice.
- **Status synchronization:** alarm states (active, acknowledged, cleared) are kept in sync with the backend so that the HMI always reflects the latest condition.
- **FINS-based event detection:** each alarm is bound directly to PLC tags transmitted over the FINS UDP protocol, which supports fast and accurate detection.

Testing shows that the system detects high-temperature events and displays the corresponding alarms with a delay of less than 200 ms after the PLC value changes. These results indicate that the developed FINS connector can reliably handle both routine data exchange and realtime event notification in the HMI.

4.4. Performance Evaluation

Performance testing combined automated benchmarks with user feedback analysis. Table 3 presents the evaluation metrics and outcomes.

Table 3: Summary of Technical Evaluation Results

Metric	Target	Observed
Read/write latency (avg.)	≤ 200 ms	12 ms
Communication success rate	$\geq 95\%$	99%
Server CPU usage	$\leq 30\%$	2%
Server memory usage	≤ 500 MB	462 MB
Alarm detection time	≤ 200 ms	1000 ms (formal)
Application startup time	≤ 10 s	3.14 s

4.4.1. Read/Write Latency

Latency measurements involved writing data to PLC memory (W1 register) followed by immediate read-back operations. Testing across 100 trials yielded an average latency of **12 ms**, significantly under the 200 ms benchmark. These measurements confirm that the connector maintains fast response times under controlled laboratory conditions.

4.4.2. Communication Success Rate

Stress testing with 1 000 requests at concurrency level 10 achieved **990 successful responses** against **10 timeouts** (99% success rate). Wireshark analysis showed missing replies for failed requests, attributed to network packet loss, PLC processing delays, or host scheduling conflicts. Successful communications exhibited proper FINS frame structure without protocol errors.

4.4.3. Resource Utilization

Load testing revealed minimal CPU usage (2–3%) with memory consumption at 91 MB (backend) and 371 MB (frontend). The Node.js and Angular components operated efficiently on standard PC hardware during testing.

4.4.4. Alarm Detection Time

Alarm detection measures the interval from PLC trigger activation to HMI display update. Measured detection time averaged 1000 ms, exceeding the 200 ms target. This delay stems from the system’s fixed 1-second polling cycle for alarm evaluation, shown in Fig. 7.

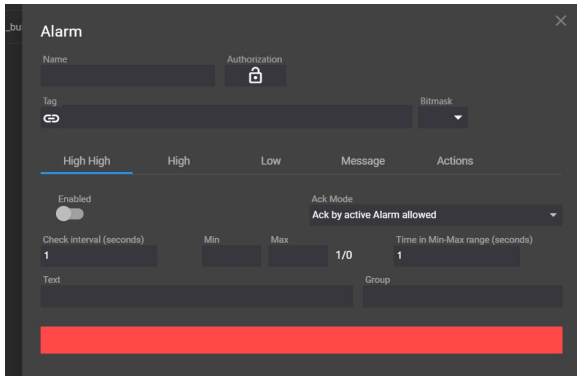


Figure 7: HMI system alarm detection interval limitation.

4.4.5. Application Startup Time

Application initialization averaged **3.14 s**, meeting the ≤ 10 s requirement and confirming adequate startup performance.

4.5. Usability Evaluation

User feedback collection involved 10 operators and technicians rating system aspects on a 5-point scale. Categories included configuration simplicity, status clarity, responsiveness, interface consistency, and overall satisfaction. Average ratings were:

- Configuration ease: 4.2
- Clarity of status: 4.1
- Responsiveness: 4.3
- UI consistency: 4.0
- Overall satisfaction: 4.3

User comments praised the clear connection status display while suggesting improvements: more visible *Check Connection* controls and simplified logging for basic troubleshooting.

4.6. Discussion

FINS connector integration with FUXA created a working HMI system supporting reliable Omron CJ2M communication under test conditions. Key findings include:

- **Reliability:** Communication success rate (99%) and correct packet framing demonstrate interoperability with Omron FINS.
- **Performance:** Read/write latencies (12 ms) and low resource usage indicate the approach is suitable for responsive HMI operations on modest hardware.

- **Alarm timeliness:** The primary area needing improvement is alarm detection latency due to the current polling-based approach. Mitigation strategies include polling prioritization, reducing polling intervals carefully, or exploring event-driven mechanisms if supported by the PLC network stack.
- **Robustness:** Observed timeouts under high-stress scenarios suggest adding retry backoff, packet pacing, and enhanced logging for root-cause analysis in noisy networks.
- **Usability:** End-user feedback is positive; additional UX improvements and operator-focused logging would increase practical adoption.

5. CONCLUSION

This research successfully implemented and evaluated the integration of the Factory Interface Network Service (FINS) protocol into the open-source Human–Machine Interface (HMI) system FUXA. The development introduced a new backend connector module (DeviceFins) written in Node.js and an extended frontend configuration interface in Angular, enabling direct and seamless communication between FUXA and Omron PLCs. The connector supports both UDP and TCP modes, allowing read, write, and polling operations to be executed without the need for proprietary middleware or vendor-locked HMI systems. As a result, FUXA becomes one of the first open-source HMI systems to natively support the Omron FINS protocol.

Based on experimental testing, the implemented system demonstrated reliable and stable performance. The communication success rate reached 99%, and the average read/write latency was measured at 12 ms, well below the 200 ms KPI target. System resource consumption remained efficient, with average CPU usage of 2% and memory consumption of 462 MB during load testing. The HMI startup time averaged 3.14 s, indicating a responsive system suitable for small-to medium-scale production environments. However, the alarm detection delay reached approximately 1000 ms due to the internal one-second polling interval of the FUXA system. This limitation shows that the alarm subsystem still relies on a time-driven polling approach rather than a fully event-driven mechanism.

Overall, the integration of FINS into FUXA successfully fulfilled the two primary research objectives: (1) enabling the HMI system to communicate with Omron PLCs using the FINS protocol

with functionality equivalent to existing Modbus support, and (2) improving the stability and visualization of data exchange on the HMI interface. Future research should focus on optimizing the alarm handling mechanism through event-driven or interrupt-based approaches, enhancing network security with TLS or VPN, and expanding multi-protocol interoperability. Additionally, open publication of the connector's source code, testing scripts, and datasets is encouraged to promote reproducibility and community-driven advancement in open-source industrial automation systems.

REFERENCES

- [1] Mohsin Ali Koondhar dkk. "The Role of PLC in Automation, Industry, and Education: A Review". In: *Pakistan Journal of Engineering Technology and Science* 11.2 (2023), pp. 22–31. doi: 10.22555/pjets.v11i2.975.
- [2] S. Mujawar. "Introduction to Human–Machine Interface". In: *Handbook/Book on Human–Machine Interfaces*. Chapter 1; Accessed via Wiley Online Library. Wiley, 2023. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394200344.ch1> (visited on 09/26/2025).
- [3] Jyotika Sawal. "SCADA Market Size, Share, Trend Analysis by 2033". <https://www.emergenresearch.com/industry-report/scada-market>. Report ID: ER_004544. May 2025. URL: https://www.emergenresearch.com/industry-report/scada-market?srltid=AfmB0orSdQPJM_IH0PBERiqxoKYb023L8TR4JyuIII93M6iZJ4ojmwiu.
- [4] Alisyn Gularte. "Open vs. Proprietary: Choosing the Right SCADA System for Your Solar Plant". Blog post, Nor-Cal Controls. Accessed: 2025-09-12. Feb. 2025. URL: <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>.
- [5] Bakul Banthia. *Understanding the Risks of Cloud Vendor Lock-In*. Disaster Recovery Journal. Tessell. Aug. 13, 2024. URL: https://drj.com/industry_news/understanding-the-risks-of-cloud-vendor-lock-in/.
- [6] McKinsey & Company. "Is industrial automation headed for a tipping point?" In: *McKinsey & Company Insights* (2023). URL: <https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point>.
- [7] Kasun Thushara. "Getting Start with FUXA - Web Based SCADA Tool". Accessed: 2025-07-16. 2024. URL: https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/.
- [8] Davide Ancona dkk. "Towards Runtime Monitoring of Node.js and Its Application to the Internet of Things". In: *arXiv preprint arXiv:1802.01790* (2018). doi: 10.48550/arXiv.1802.01790. URL: <https://arxiv.org/abs/1802.01790>.
- [9] Joshua D. Scarsbrook, Mark Utting, and Ryan K. L. Ko. "TypeScript's Evolution: An Analysis of Feature Adoption Over Time". In: *arXiv preprint arXiv:2303.09802* (2023). URL: <https://arxiv.org/abs/2303.09802>.
- [10] Peter Humaj. "Communication with Omron FINS". Blog post on Ipesoft D2000. Apr. 2021. URL: <https://d2000.ipesoft.com/blog/communication-omron-fins/>.
- [11] Brijesh Joshi. "A Study of the Various Communication Protocols Used in PLC Systems: Modbus, Profibus, Ethernet/IP and Their Implementations, Evolution and Comparative Analysis". In: *International Research Journal of Modernization in Engineering Technology and Science (IRJMETS)* 6.4 (Apr. 2024). doi: 10.56726/IRJMETS52882. URL: https://www.irjmets.com/uploadedfiles/paper/issue_4_april_2024/52882/final/fin_irjmets1713198034.pdf.
- [12] Sheikh Usman Uddin, Mirza Jabbar Aziz Baig, and Mohammad Tariq Iqbal. "Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System". In: *Sensors* 22.24 (2022), p. 9631. doi: 10.3390/s22249631. URL: <https://www.mdpi.com/1424-8220/22/24/9631>.
- [13] Mohammad Omid, Majid Salehifar, and Amir Mohammadi. "Design and Implementation of an Open Source SCADA System for Monitoring a Hybrid Renewable Power System".

- In: *Energies* 16.5 (2023), p. 2229. DOI: 10.3390/en16052229. URL: <https://www.mdpi.com/1996-1073/16/5/2229>.
- [14] M. S. Almas and L. Vanfretti. “Open Source SCADA Implementation and PMU Integration”. In: *IEEE PES General Meeting* (2014).
 - [15] Luis Salazar dkk. “ICSNet: A Hybrid-Interaction Honeynet for Industrial Control Systems”. In: *Proceedings of the Sixth Workshop on CPS&IoT Security and Privacy (CPSIoTSec’24)*. Salt Lake City, UT, USA: ACM, 2024, pp. 1–12. ISBN: 979-8-4007-1244-9. DOI: 10.1145/3690134.3694813. URL: <https://doi.org/10.1145/3690134.3694813>.
 - [16] Robby Hazdi, Muhammad Fadli, and Yusril Maulana. “Perancangan SCADA pada Sistem Otomatisasi Rumah”. In: *eProceedings of Applied Science* 3.2 (2017). Universitas Telkom, pp. 1099–1106. URL: <https://openlibrarypublications.telkomuniversity.ac.id/index.php/appliedscience/article/view/4046>.